

#1.Array Manipulation Function

```
In [1]: import numpy as np
```

```
In [6]: a = np.array([1,2,3,4])           #reshape
        reshaped = np.reshape(a,(1,4))
        print("Reshaped array :",reshaped)
```

Reshaped array : [[1 2 3 4]]

```
In [7]: a = np.array([1,2,3,4])
        reshaped = np.reshape(a,(4,1))
        print("Reshaped array :\n",reshaped)
```

Reshaped array :

```
[[1]
 [2]
 [3]
 [4]]
```

```
In [10]: b = np.array([[4,5,9],[6,7,8]])    #ravel
        flattened = np.ravel(b)
        print("Flattened array :",flattened)
```

Flattened array : [4 5 9 6 7 8]

```
In [12]: c = np.array([[4,5,9],[6,7,8]])    #transpose
        transposed = np.transpose(c)
        print("Transposed array :\n",transposed)
```

Transposed array :

```
[[4 6]
 [5 7]
 [9 8]]
```

```
In [13]: a1 = np.array([1,2])
        b1 = np.array([3,4])
        c1 = np.array([5,6])
        stacked = np.vstack([a1,b1,c1])    #vstack
        print("Stacked arrays :\n",stacked)
```

Stacked arrays :

```
[[1 2]
 [3 4]
 [5 6]]
```

2.Mathematical Functions

```
In [19]: a1 = np.array([1,2,4,5])
        added = np.add(a1, 3)           #add 3 to each element
        print("Added 3 to a1 :",added)
```

Added 3 to a1 : [4 5 7 8]

```
In [20]: b1 = np.array([1,2,4,9])
        added = np.add(b1, 9)
```

```
print("Added 9 to b1 :",added)
```

Added 9 to b1 : [10 11 13 18]

```
In [22]: a2 = np.array([1,2,4,5])
sub = np.subtract(a2, 3)      #sub -3 to each element
print(" subtract -3 to a2 :",sub)
```

subtract -3 to a2 : [-2 -1 1 2]

```
In [23]: a2 = np.array([1,2,4,5])
squared = np.power(a2,2)     # square each element
print("square of a2:",squared)
```

square of a2: [1 4 16 25]

```
In [25]: a2 = np.array([1,2,4,5])
sqrt_value = np.sqrt(a2)    # square root each element
print("square root of a2:",sqrt_value)
```

square root of a2: [1. 1.41421356 2. 2.23606798]

```
In [26]: print(a1)
print(a2)
```

[1 2 4 5]

[1 2 4 5]

```
In [27]: b1 = np.array([1,2,4,9])
dot_product = np.dot(a1,a2)  #dot product
print("Dot product of a1 and a2 :",dot_product)
```

Dot product of a1 and a2 : 46

3.Statistical functions

```
In [29]: s = np.array([1,2,3,4,5,6])
mean = np.mean(s)           #mean
print("Mean of s:",mean)
```

Mean of s: 3.5

```
In [30]: s
```

Out[30]: array([1, 2, 3, 4, 5, 6])

```
In [31]: std_dev = np.std(s)      #standard deviation
print('Standard deviation of s :',std_dev)
```

Standard deviation of s : 1.707825127659933

```
In [32]: minimum = np.min(s)     #minimum element
print('Minimum value of s:',minimum)
```

Minimum value of s: 1

```
In [33]: maximum = np.max(s)     #maximum element
print('Maximum value of s:',maximum)
```

Maximum value of s: 6

4.Linear Algebra Function

```
In [34]: matrix = np.array ([[1,2],[3,9]])      # create a mtrix
print(matrix)
```

```
[[1 2]
 [3 9]]
```

```
In [36]: matrix1 = np.array ([[1,2],[3,9],[7,8]])      # create a mtrix
print(matrix1)
```

```
[[1 2]
 [3 9]
 [7 8]]
```

5.Random Sampling Functions

```
In [37]: random_vals = np.random.rand(3)  #generate array of 3 random values between 0 and 1
print("Random values:",random_vals)
```

```
Random values: [0.43374865 0.44190744 0.44799204]
```

```
In [38]: random_vals = np.random.rand(5)  #generate array of 5 random values between 0 and 1
print("Random values:",random_vals)
```

```
Random values: [0.51160907 0.45652687 0.87804287 0.09043734 0.02287366]
```

```
In [39]: np.random.seed(0)      # set seed for reproducibility
random_vals = np.random.rand(3)  #generate 3 random values between 0 and 1
print("Random values:",random_vals)
```

```
Random values: [0.5488135  0.71518937 0.60276338]
```

```
In [41]: rand_ints = np.random.randint(0,10,size = 5)  #generate array of 5 random integers
print("Random integers:",rand_ints)
```

```
Random integers: [2 4 7 6 8]
```

```
In [43]: np.random.seed(0)      # set seed for reproducibility
rand_ints = np.random.randint(0,10,size = 5)  #generate array of 5 random integers
print("Random integers:",rand_ints)
```

```
Random integers: [5 0 3 3 7]
```

6. Boolean & Logical Functions

```
In [44]: # check if all elements are True
#all
logical_test = np.array([True,False,True])
all_true = np.all(logical_test)
print("All elements true:",all_true)
```

```
All elements true: False
```

```
In [4]: import numpy as np
```

```
In [5]: logical_test = np.array([True,True,True])
all_true = np.all(logical_test)           #all()
print("All elements true:",all_true)
```

All elements true: True

```
In [6]: logical_test = np.array([True,False,False])
any_true = np.any(logical_test)           #any()
print("any elements true:",any_true)
```

any elements true: True

```
In [7]: logical_test = np.array([False,False,False])
any_true = np.any(logical_test)           #any()
print("any elements true:",any_true)
```

any elements true: False

7.Set Operation

```
In [10]: set_a = np.array([1,7,3,9])
set_b = np.array([2,9,3,0])
Intersection = np.intersect1d(set_a,set_b)   #intersection
print('Intersection of a and b:',Intersection)
```

Intersection of a and b: [3 9]

```
In [11]: set_a = np.array([1,7,3,9])
set_b = np.array([2,9,3,0])
Union = np.union1d(set_a,set_b)             #union
print('Union of a and b:',Union)
```

Union of a and b: [0 1 2 3 7 9]

8.Array Attribute Function

```
In [13]: arr = np.array([[1,2,3],[4,5,7]])
print(arr)
```

```
[[1 2 3]
 [4 5 7]]
```

```
In [15]: Shape = arr.shape           #shape of the array
print(" Shape of array = ",Shape)
```

Shape of array = (2, 3)

```
In [16]: Size = arr.size              #number of elements
print("Size of array = ",Size)
```

Size of array = 6

```
In [18]: Dimention = arr.ndim         #number of dimentions
print('Dimention of array = ',Dimention)
```

Dimention of array = 2

```
In [19]: dtype = arr.dtype           #datatype of array
print('Datatype of array = ',dtype)
```

Datatype of array = int64

9.Other functions

```
In [20]: arr = np.array([[1,2,3],[4,5,7]])  
copied_arr = np.copy(arr)      #create a copy of array a  
print('copied array =\n',copied_arr)
```

```
copied array =  
[[1 2 3]  
 [4 5 7]]
```

```
In [21]: array_size_in_bytes = arr.nbytes  # size in bytes of array  
print('Size of arr in bytes =',array_size_in_bytes)
```

Size of arr in bytes = 48

```
In [22]: shared = np.shares_memory(arr,copied_arr)    #check if arrays share memory  
print('Do arr and copied_arr share memory?',shared)
```

Do arr and copied_arr share memory? False

```
In [ ]:
```